PROGRAMAÇÃO MOBILE II

Sistema de Conta Bancária

Estudo de Caso Prático em Kotlin

Aplicando os conceitos essenciais da linguagem em um domínio real

val / var	Null Safety	when	Funções	HOF

Duração estimada	Modalidade	Pré-requisitos
90 minutos	Individual ou dupla	Aula teórica de Kotlin básico

Disciplina: Programação Mobile II · Linguagem: Kotlin · Plataforma: Android

1. Contexto do Problema

Uma startup de fintechs contratou você para desenvolver o módulo central de gerenciamento de contas correntes do seu aplicativo Android. O módulo deve ser robusto, seguro contra erros de nulo e fácil de manter. Seu líder técnico estabeleceu as seguintes exigências:

R1	Dados imutáveis (ex.: número da conta) nunca devem ser reatribuídos após a criação.
R2	O campo 'titular' é opcional — o sistema não pode travar se ele for nulo.
R3	Transações (depósito, saque, transferência) devem ser tratadas de forma expressiva.
R4	A função de transação deve aceitar uma taxa opcional, com padrão de 0%.
R5	Regras de negócio (ex.: impostos) devem ser aplicadas via funções de ordem superior.

> Objetivo: implementar o módulo em Kotlin, respeitando todas as 5 exigências e demonstrando domínio dos conceitos estudados.

Conceitos utilizados neste estudo de caso

val / var	Controle de mutabilidade — dados fixos vs. dados que mudam ao longo do tempo
Null Safety (?. !! ?:)	Operadores para lidar com valores opcionais sem lançar NullPointerException
when	Estrutura de controle de fluxo expressiva, substituta do switch/case
Parâmetros nomeados e padrão	Funções mais legíveis e flexíveis, sem necessidade de sobrecarga
Higher-Order Functions	Funções que recebem outras funções como argumento — base do Compose

2. Estrutura Base Fornecida

O código abaixo é o esqueleto do módulo. Ele já está parcialmente implementado. Sua tarefa é completar as partes marcadas com **TODO** em cada atividade.

Arquivo: ContaBancaria.kt

```kotlin
// Estrutura base — ContaBancaria.kt
// Programação Mobile II

// ① Declaração de variáveis
val numeroConta: String = "CC-2024-001"
var saldo: Double = 1500.00
var titular: String? = null // campo opcional

// ② Função principal de transação (a ser completada)
fun processarTransacao(
valor: Double,
tipo: String,
taxa: Double = 0.0 // TODO atividade 3: entender este padrão
): Double {
// TODO atividade 3: implementar o corpo
return saldo
}
```

```kotlin

// ③ Função de ordem superior (a ser completada)
fun aplicarRegra(saldo: Double, regra: (Double) -> Double): Double {
// TODO atividade 5: implementar o corpo
return saldo
}
```

3. Atividades

1 val e var — Imutabilidade e Mutabilidade

Analise a estrutura base e responda: por que **numeroConta** foi declarado com **val** e **saldo** com **var**? O que acontece se você tentar reatribuir **numeroConta** depois da inicialização?

1a) Explique a diferença entre val e var com suas próprias palavras:

Usamos val quando o valor não vai ser re-atribuido, não mutável. Já var é mutável

1b) Complete o código abaixo adicionando mais um campo imutável (**agencia**) e um campo mutável (**limiteChequeEspecial**):

```
// Adicione as declarações abaixo:

val agencia: String = "0042"

var limiteChequeEspecial: Double = 500.0
```

1c) Por que limiteChequeEspecial deve ser var e não val?

Pois sofrerá mudanças

2 Null Safety — Operadores ?. !! e ?:

O campo **titular** é do tipo **String?** (anulável). Você precisa exibir o nome em maiúsculas na tela do app. Se o titular for nulo, exiba *"Cliente não identificado"*.

2a) Complete a linha usando o operador safe call (?.) e o operador Elvis (?:):

```
val nomeExibido: String = titular?__.uppercase() ?:__"Cliente não identificado"
```

```
println("Bem-vindo, $nomeExibido")
```

2b) O que ocorre se você usar !! em vez de ?. e o titular for nulo? Quando é aceitável usar !! no código de produção?

xzlxlz

A nullPointerException será exibido

> Dica: ?. executa a chamada somente se o objeto não for nulo. ?: fornece um valor padrão quando a expressão à esquerda é nula.

3 when + Parâmetros Nomeados e Padrão

Implemente o corpo da função **processarTransacao** da estrutura base. Use **when** para tratar cada tipo de transação e aplique a taxa sobre o valor antes de calcular o novo saldo.

3a) Complete a implementação:

```
fun processarTransacao(
valor: Double,
tipo: String,
taxa: Double = 0.0
): Double {
val valorLiquido = valor - (valor * taxa / 100)
return when (tipo) {
"deposito" -> saldo + valorLiquido____
"saque" -> saldo - valorLiquido_______
"transferencia" -> saldo - valorLiquido______
else -> saldo // tipo inválido: retorna saldo atual
}
}
```

3b) Faça as chamadas abaixo usando parâmetros nomeados e teste os dois cenários:

```
// Cenário 1: depósito de R$ 500 sem taxa
saldo = processarTransacao(valor = 500.0, tipo = “deposito”___)
println("Saldo após depósito: R$ $saldo") // esperado: R$ 2000.0
```

```
// Cenário 2: saque de R$ 200 com taxa de 2,5%
saldo = processarTransacao(valor = 200.0, tipo = "saque"_____, taxa = 2.5__)
println("Saldo após saque: R$ $saldo") // esperado: R$ 1805.0
```

3c) Qual é a vantagem de usar parâmetros nomeados na chamada da função?

Tornam mais legível e autodocumentada

4 Função de Expressão Única + when como Expressão

Converta a função **descricaoTransacao** para a forma de **expressão única** (usando **=** em vez de bloco **{ return }**). Em seguida, escreva também a versão como **função local** dentro de main.

4a) Versão tradicional (já dada):

```
fun descricaoTransacao(tipo: String): String {
return when (tipo) {
"deposito" -> "Entrada de valores"
"saque" -> "Retirada de valores"
"transferencia" -> "Movimentação entre contas"
else -> "Tipo desconhecido"
}
}
```

4b) Reescreva como função de expressão única:

```
fun descricaoTransacao(tipo: String): String = when (tipo) {
```

```
"deposito" -> "Entrada de valores"________
"saque" -> "Retirada de valores"_______
"transferencia" -> "Movimentação entre contas"_
else -> "Tipo desconhecido"_________
}
```

4c) Qual benefício essa forma traz para a legibilidade e manutenção do código?

Deixa mais conciso com foco na lógica de retorno

5 Funções de Ordem Superior (Higher-Order Functions)

Complete a função **aplicarRegra** e use-a para aplicar dois tipos de regra de negócio ao saldo: uma isenção de taxa e um imposto sobre rendimento. Passe as regras como **lambdas**.

5a) Complete a função e as chamadas:

```
fun aplicarRegra(saldo: Double, regra: (Double) -> Double): Double {
return regra(saldo)____// aplique a regra ao saldo e retorne
}

// Regra 1: imposto de 15% sobre o saldo
val saldoComImposto = aplicarRegra(saldo) { s -> s - (s * 0.15)__}

// Regra 2: bônus de fidelidade de 5%
val saldoComBonus = aplicarRegra(saldo) { s -> s + (s * 0.05)__}

println("Com imposto : R$ ${"%.2f".format(saldoComImposto)}")
println("Com bônus : R$ ${"%.2f".format(saldoComBonus)}")
```

5b) Explique com suas palavras o que é uma Higher-Order Function e cite um exemplo de uso no Jetpack Compose:

Trata-se de função que retorna outra função ou recebe uma de parametro, um exemplo é onClick em Button.

6 Desafio — Integração Completa

Integre todos os conceitos em uma função **main** funcional. A saída esperada está descrita abaixo. Escreva o código completo no espaço disponível.

Saída esperada ao executar main():

```
Conta : CC-2024-001 | Agência: 0042
Titular : ALICE
■■■
```